

HomeFront Application Architecture

Technical Code Walkthrough & Database Interactions

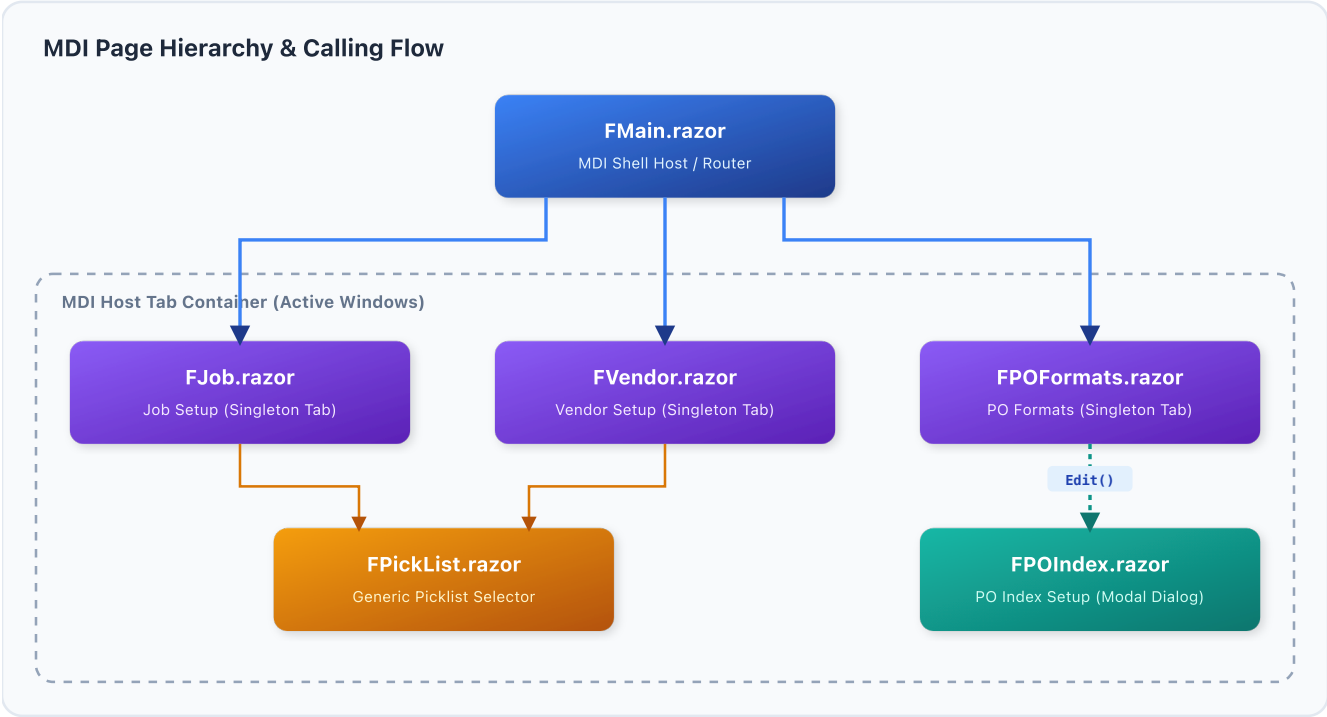
A comprehensive architecture report and program walkthrough analyzing the Multiple Document Interface (MDI) system inside `FMain.razor`, and the functional logic and database queries of `FJob.razor` (Job Setup), `FVendor.razor` (Vendor Setup), and `FP0Index.razor` (Purchase Order Index Setup).

1. Executive Summary & Architecture Overview

The HomeFront migrated application implements a robust **Multiple Document Interface (MDI)** container shell within Blazor Server. It replicates the core visual fidelity, behavioral parity, and tabbed workflow navigation of the legacy VB6 client. Pages are hosted dynamically within the main layout and share global context while preserving visual visual state and avoiding unnecessary reload cycles.

This walkthrough analyzes the host shell `FMain.razor` and the following three principal components:

- **Job Setup (`FJob.razor`)**: The primary workspace for defining projects, models, phase listings, custom property categories, personnel assignments, and purchase orders. Hosted as a singleton tab.
- **Vendor Setup (`FVendor.razor`)**: The profile workspace for managing vendor metadata, community-specific tax configurations, payment percentage points, contacts, and compliance insurance requirements. Hosted as a singleton tab.
- **PO Index Setup (`FP0Index.razor`)**: A modular overlay dialog called by the PO Formats interface to create templates, assign default cost codes, categories, and documents.



2. FMain.razor — The MDI Shell Container

`FMain.razor` acts as the root coordinator. Rather than allowing standard browser page navigation which disrupts visual continuity and wipes page states, `FMain` intercepts URL routing changes using a Blazor-native `NavigationManager` handler:

```
_navInterceptor = NavigationManager.RegisterLocationChangingHandler(OnLocationChanging);
```

When a child page initiates a routing call (e.g. `/job-setup`, `/vendor`), the interceptor performs the following:

1. Checks if the requested route is registered in its routing table (`_routeMap`). If so, it invokes `context.PreventNavigation()` to block default browser behavior.
2. Delegates the call internally to `NavigateToPage(relativePath)`.
3. If the target is configured as a **Singleton**, it searches the active window list:

```
var existing = _windows.FirstOrDefault(w => w.PageType == pageType);
```

If found, it focuses the existing tab by updating `_activeWindowId`. If not, it instantiates a new `MdiWindow` container, registers it in `_windows`, and activates it.

4. The active window is rendered dynamically in the layout using `MdiHost`:

```
<MdiHost Windows="@_windows" ActiveWindowId="@_activeWindowId" ... />
```

Singleton Classification

Because `_multiInstanceRoutes` only contains `/edit-model-assembly` (`FAssembly`), both **FJob** and **FVendor** operate as Singletons. Only one tab for each page is active at any time. When a user changes the active record (e.g. switching jobs), the component publishes a refined state, updating the active tab parameters in-place rather than opening new tabs.

3. FJob.razor — Job Setup Deep Dive

FJob.razor manages job setup records, project details, and default parameters. It binds to the route `/job-setup` or `/job-setup/{JobNumber}`.

Functional Workflow & Fidelity

- **Automatic Picklist Prompt:** If opened with no active job loaded (e.g., initial open or session clear), it pops the `FPickList` during `OnAfterRenderAsync`, querying `tbljobs` so the page does not render empty.
- **Properties Tree Grid:** Embeds a `TreeGridControl` to represent legacy VB6 outline trees. It groups scheduling parameters, tax groups, and Tarion warranty items. Double-clicking values opens in-line input text boxes, checkboxes, or dropdown selectors.
- **User-Defined Fields:** Dynamically reads field definitions from `JobCustomFieldDefs` and reads/writes values from the custom table `JobCustomFields`.
- **Personnel & Contacts:** Displays two separate grids: `Personnel` (PM, Estimator, Purchaser loaded from `tblProjectManager`) and `Contacts` (roles loaded from the division-scoped option `JobContactRoles`).

Database Interactions & SQL Queries

Operation	Database SQL Statement	Description
Query Options	<pre>SELECT TOP 1 * FROM system_setup WHERE id=@Id SELECT TOP 1 optionvalue FROM appoptions...</pre>	Loads country setting, accounting/sales system enums, and precon/Tarion flag states.
Load Job Record	<pre>SELECT j.*, c.ar_customer_deposit FROM tbljobs j LEFT OUTER JOIN tblcustomers c ON j.job_no=c.job_no WHERE j.DivisionID=@DivisionID AND j.job_no=@Job</pre>	Pulls primary job parameters, description, municipal address, status, community, and model links.
Load Personnel	<pre>SELECT 'PM' ContactType, p.* FROM system_setup s LEFT JOIN tblJobs j... LEFT JOIN tblProjectManager p... UNION ALL...</pre>	Runs a three-part UNION query to populate the PM, Estimator, and Purchaser records for the active job.
Load POs Grid	<pre>SELECT p.PONumber, p.PODate, p.POIndex, SUM(d.pretax) Pretax, SUM(d.jctax + d.njctax) Tax... FROM pomaster p LEFT JOIN tblvendors v... LEFT JOIN poitems d... GROUP BY...</pre>	Aggregates purchase order numbers, vendors, pretax, tax, totals, approvals, and cancellation states.

Operation	Database SQL Statement	Description
Insert New Job	<pre>INSERT INTO tblcustomers (..., Customer_No, Job_no, bal_sheet_Prefix, community, phase...) VALUES (...)</pre>	Creates a new Customer record (which triggers automatic job creation inside the database schema).
Update Job	<pre>UPDATE tblJobs SET Description=@Description, Notes=@Notes, Municipal_Address=@Address, Inactive=@Inactive, Lot=@Lot, Model=@Model, PermitNumber=@PermitNumber, ScheduleTemplate=@ScheduleTemplate... WHERE Job_No=@Job AND DivisionID=@DivisionID</pre>	Saves primary job text fields, date values, drop-down selections, and tax group mappings.
Save Custom Fields	<pre>UPDATE JobCustomFields SET [FieldName]=@cf0... WHERE divisionid=@DivisionID AND Job_No=@Job</pre>	Saves user-defined custom properties based on fields loaded from JobCustomFieldDefs .

4. FVendor.razor — Vendor Setup Deep Dive

FVendor.razor provides vendor profiles. It binds to the route /vendor or /vendor/{VendorIdParam} .

Functional Workflow & Fidelity

- **Initial Prompt:** Replicates the VB6 behavior by immediately popping the vendor picklist overlay on page load if no vendor has been selected yet.
- **Trade Group & PO Format Dropdowns:** Trade groups are populated dynamically from existing records (SELECT DISTINCT VendorGroupID...). PO Formats scan report template files (*.rpt) inside wwwroot/resources/Estimating/PO Formats/ .
- **Grids Layout (IniGetGrid):** Columns for Contacts (gContacts), Payment Points (gPayPoints), and Insurance (gInsurance) are rendered dynamically from AppGridLayout configurations via the shared GridLayoutPresenter .
- **Tax Groups Defaults:** The Tax Groups grid displays a fixed first row representing default group mappings stored in tblVendors (e.g. Material, Labour, Subcontract). Subsequent rows map community-specific tax mappings from vendortaxgroups .

Database Interactions & SQL Queries

Operation	Database SQL Statement	Description
Load Vendor Record	<pre>SELECT wp.walletid AS WalletPayeeID, v.* FROM tblVendors v JOIN divisions d... LEFT JOIN wallet_payees wp... WHERE v.Vendor_ID = @VendorId AND v.divisionid = @DivisionId</pre>	Loads primary address, phone, web credentials, trade type, tax ID, and holdback percent values.
Load Tax Groups	<pre>SELECT t.community, c.description, t.MaterialTaxGroup, t.LabourTaxGroup... FROM vendortaxgroups t JOIN tbllocality c ON t.Community = c.area WHERE t.vendor = @Vendor</pre>	Reads community-specific tax structures linked to localities (tbllocality).
Load Contacts	<pre>SELECT * FROM contacts WHERE DivisionID = @DivisionID AND contacttypeid=99 AND vendorcode = @Vendor ORDER BY firstname</pre>	Loads vendor contacts, cell numbers, fax, emails, and purchasing/field/scheduling notification options.

Operation	Database SQL Statement	Description
Update Vendor	<pre>UPDATE tblVendors SET HoldbackPercentage=@Holdback, Vendor_Name=@VendorName, Addr1=@Addr1, isTBD=@IsTBD, webuid=@WebUID, wcinsrequired=@WcReq, wcinsCompany=@WcCo... WHERE Vendor_ID = @VendorId AND divisionid = @DivisionId</pre>	Updates primary vendor records, insurance criteria (expiry dates, policies, underwriters), and default tax values.
Save Pay Points	<pre>IF NOT EXISTS(SELECT 1 FROM VendorPaymentPoints WHERE...) INSERT INTO VendorPaymentPoints...; UPDATE VendorPaymentPoints SET PayPoint1Percent=@P1...</pre>	Performs a merge-style upsert of contract payment percentage configurations mapped to PO Indexes.
Sync Unsent POs	<pre>UPDATE p SET DeliveryMethod = v.PurchDelMethod, DeliveryRecipient = v.PurchContact... FROM tblvendors v JOIN pomaster p... AND p.deliverydate IS NULL WHERE v.vendor_id = @VendorId</pre>	Pushes modified vendor PO-delivery preferences to all unsent purchase orders in the database.

5. FP0Index.razor — PO Index Setup Deep Dive

Unlike `FJob` and `FVendor`, `FP0Index.razor` has no standalone route mapping. It operates as a modal child component embedded in the PO Formats configuration screen (`FP0Formats.razor`):

```
<FP0Index @ref="poIndexEditor" OnClosed="OnPOIndexClosed" />
```

Functional Workflow & Calling Flow

- **Async Dialog Invocation:** To configure an index, the parent screen invokes the public `Edit(poIndex)` method:

```
bool saved = await poIndexEditor.Edit("MATERIAL_PO");
```

This method returns a `Task<bool>` that suspends execution of the parent page while the user interacts with the modal dialog, returning `true` if changes are saved.

- **Fidelity Details:** Includes standard fields for FOB, Ship Via, Terms, Cost Codes, and Categories. Options checkboxes include "Measurement PO", "Warranty PO", "Precon PO", and "Send to BuildPro".
- **Document Linkage:** Displays a checklist of attachment types (e.g. Subcontracts, Lien Releases) derived from `Attachments` and maps selections to `POIndexDocuments`.

Database Interactions & SQL Queries

Operation	Database SQL Statement	Description
Load PO Index Data	<pre>SELECT * FROM tblPOIndex WHERE DivisionID = @DivisionID AND POIndex = @POIndex</pre>	Loads terms, notes, retainage %, cost codes, category mappings, and system settings.
Load Attachments	<pre>SELECT DISTINCT j.DocumentClass, CASE WHEN p.Class IS NULL THEN 'False' ELSE 'True' END AS Selected FROM Attachments j LEFT OUTER JOIN POIndexDocuments p ON (...) WHERE...</pre>	Fetches the complete set of unique document attachment classes and flags if they are linked to the index.
Update PO Index	<pre>UPDATE tblPOIndex SET Description=@Description, StandardText=@StandardText, RetainagePercent=@RetainagePercent, JCCostCode=@JCCostCode, JCCategory=@JCCategory, WarrantyPO=@WarrantyPO... WHERE</pre>	Updates index settings, retainage values, and BuildPro validation flags.

Operation	Database SQL Statement	Description
	DivisionID=@DivisionID AND POIndex=@OldPOIndex	
Save Attachments	DELETE FROM POIndexDocuments WHERE POIndex=@POIndex INSERT INTO POIndexDocuments(POIndex, Class) VALUES(...)	Rewrites document mappings inside POIndexDocuments .
Update Un-issued Retainage	UPDATE JobPOIndex SET RetainagePercent=@RetainagePercent FROM EstimateItems e JOIN JobPOIndex j ON (...) WHERE e.POGenBatch = 0 AND e.POIndex = @POIndex	Applies updated retainage percentages to all unissued estimate items matching the PO Index.
Cascade Rename	UPDATE CustomPreEstimateItems SET POIndex=@New... UPDATE EstimateItems SET POIndex=@New... UPDATE JobPOIndex SET POIndex=@New... ... (8 tables total)	Propagates index code updates across all referencing tables in the database (runs inside an On Error Resume Next block parity).

6. Database Interaction Matrix

The following matrix maps the primary database tables accessed by **FMain** , **FJob** , **FVendor** , and **FP0Index** , illustrating the read/write (CRUD) operations:

Table Name	FMain	FJob	FVendor	FP0Index	Purpose
tblJobs	R	C / R / U	-	-	Job setup profiles and scheduling properties.
tblcustomers	-	C / R / U	-	-	Main buyer and single-family selection mappings.
arcustomers	-	C / R	-	-	Accounts Receivable accounts linked to jobs.
tblVendors	-	R	R / U	-	Vendor metadata and default insurance fields.
tblPOIndex	-	-	-	R / U	Purchase order setup templates.
POIndexDocuments	-	-	-	C / R / U / D	Attachments linked to PO Index setups.
VendorPaymentPoints	-	-	C / R / U	-	Contract payment percentage stages.
vendortaxgroups	-	-	C / R / U / D	-	Community-specific vendor tax group overrides.
contacts	-	R	C / R / U	-	Primary company and vendor contact details.
jobContacts	-	C / R / U / D	-	-	Contact roles assigned to specific construction jobs.
JobCustomFields	-	C / R / U	-	-	User-defined metadata values for Job pages.
AppOptions / appoptions	R	R	R	R	Global and division-scoped application configuration.

7. Sibling Synchronization & Compliance

As a developer working in the HomeFront (PB, Mobile Source) and FlexKit/FlexCore codebase, the following architectural guidelines must be adhered to:

- **Fidelity Parity:** Parity to VB6 in look, feel, and behavior is paramount. Check the matching `.frm` or `.bas` files before writing or troubleshooting pages. Do not suppress errors silently; render visible fallback labels if configuration properties are missing.
- **FlexKit Constraints:** Raw HTML inputs are prohibited; all controls must resolve to `Fx.ControlKit.*` components. FlexKit controls (except `TextBoxControl`) do not forward arbitrary HTML attributes (e.g. inline style/class overrides); use built-in component properties.
- **Minimal JavaScript:** Lean on Blazor-native components (`@ref`, `@bind`, `FocusAsync`) and standard CSS selectors. JavaScript must be isolated within lazily-loaded modules with C# fallbacks.
- **Sibling Sync (Kept in Sync):** Library code files inside FlexKit and FlexCore must be identical. Sibling host pages must be mirrored between `HomeFront` (client, path `Components/Pages/Migrated/`) and `HomeFrontPB` (active builder host, path `Components/Pages/`).
- **No Database Schema Changes:** Schema migrations must never execute from the application layer. Schema edits are managed exclusively by dedicated database scripts.

Documentation Integrity

Maintain docstrings and structural comments. Run non-incremental compiles on both solutions (`HomeFront.sln` and `HomeFrontPB.sln`) after libraries are modified to avoid compilation regressions.